



PROYECTO FINAL (AE3) — CLÚSTER MPI Y MINERÍA DISTRIBUIDA

HITO 2 — Perfilado de Código (Profiling) y Límites del Sistema

Grupo

Asignatura

ESTRUCTURA DE COMPUTADORES, PARALELISMO Y SISTEMAS DISTRIBUIDOS
(GRADO EN CIBERSEGURIDAD)

1. Objetivo del Hito

En este segundo hito se profundizará en el análisis del minero construido previamente:

- Comprender por qué un solo núcleo no es suficiente y aprender a medir el rendimiento en Python.
- Analizar el código con herramientas de profiling.
- Estudiar el cuello de botella del programa.
- Justificar la necesidad de paralelización.

2. Contexto

El Hito 1 sirvió para establecer la línea base experimental del proyecto. Antes de escalar la solución, es necesario realizar un análisis para descubrir qué parte del código consume más tiempo. Entendiendo las limitaciones del hardware y del software secuencial se puede diseñar una estrategia distribuida eficiente.

3. Parte I — Perfilado de Código (Profiling) en Python

El análisis interno del código requiere que cada grupo identifique de forma precisa dónde se invierte el tiempo de CPU.

- Usar herramientas como cProfile o timeit en Python.
- Descubrir qué parte del código consume más tiempo.

4. Parte II — Monitorización de Recursos del Sistema

De forma paralela al análisis interno del código, es necesario observar cómo el sistema operativo gestiona el proceso en ejecución.

- Analizar el uso de CPU con herramientas del sistema operativo (como htop).



PROYECTO FINAL (AE3) — CLÚSTER MPI Y MINERÍA DISTRIBUIDA HITO 2 — Perfilado de Código (Profiling) y Límites del Sistema

Grupo

Asignatura

ESTRUCTURA DE COMPUTADORES, PARALELISMO Y SISTEMAS DISTRIBUIDOS
(GRADO EN CIBERSEGURIDAD)

5. Entregables

El grupo debe entregar un documento que refleje sus investigaciones:

- Breve informe identificando el cuello de botella del código secuencial.

6. Preparación para el Hito 3

Una vez comprendidas las limitaciones actuales, en el siguiente hito:

- Se realizará la introducción a mpi4py y a la paralelización estática.
- Se dará el salto a la ejecución multiproceso dentro de la misma máquina virtual.

Anexo: Herramientas y Recursos Recomendados

1. Perfilado de código con cProfile

cProfile es una herramienta integrada en Python que mide cuántas veces se llama a cada función y cuánto tiempo tarda.

```
python3 -m cProfile -s tottime minero_secuencial.py --dificultad 4 --max 1000000
```

¿Cómo leer la salida? Presta especial atención a estas columnas:

- **ncalls**: Número de veces que se ha llamado a esa función.
- **tottime**: Tiempo total de CPU invertido *dentro* de esa función.
- **cumtime**: Tiempo acumulado en esa función y en todas las subfunciones que llama.

2. Documentación Oficial de Referencia

Se recomienda consultar:

- **Documentación de cProfile y profile:**
<https://docs.python.org/3/library/profile.html>
- **Documentación del módulo timeit:**
<https://docs.python.org/3/library/timeit.html> (Útil si para aislar la función del cálculo del hash en lugar de perfilar todo el script).